МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №6

з дисципліни «Професійний Пайтон»

на тему: «Автоматизоване тестування Python-програм»

Виконав:
студент групи ФеП-32
Накладюк Володимир Тарасович
Варіант №15

Львів - 2026

1. Мета роботи

Сформувати практичні навички професійного автоматизованого тестування Python-програм за допомогою pytest: проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності (tmp_path, monkeypatch, Mock, AsyncMock) і аналізувати statement та branch coverage.

2. Індивідуальне завдання

Варіант 15. Система обліку результатів наукових експериментів. Протестувати:

- створення Experiment, Measurement, Researcher та ExperimentResult;
- валідацію parameter/value/unit/date;
- додавання результату, пошук експериментів за researcher;
- обчислення average value, визначення min/max, фільтрацію результатів за range;
- порожню ExperimentSeries, boundary та invalid measurement;
- CSV import measurements, malformed numeric record та custom exception;
- JSON export і configuration allowed parameters/units;
- ExperimentRepository через Mock/fake, analysis strategy/exporter через Mock із failure path;
- async measurement source через AsyncMock та integration experiment pipeline.

3. Короткий опис тестованого проєкту

Тестується проєкт experiment-registry, створений у лабораторних №1–5: доменна модель (value objects, сутності, generic repository, стратегії аналізу, сервіси), модуль імпорту/експорту з YAML-конфігурацією, потоковий конвеєр і консольний застосунок з підкомандами. Для вимоги про async dependency до доменного шару додано `AsyncMeasurementSource` (Protocol) і `ExperimentPipeline`, що паралельно отримує вимірювання з асинхронного джерела з обмеженням через `asyncio.Semaphore`.

Лістинг 6.1. Асинхронний конвеєр збору вимірювань

```
class ExperimentPipeline:
    """Fetches measurements concurrently (bounded by a semaphore) and records them."""

    def __init__(self, source: AsyncMeasurementSource, service: ExperimentService,
                 max_concurrency: int = 4, clock: Callable[[], date] = date.today) -> None:
        if max_concurrency < 1:
            raise ValueError("max_concurrency must be >= 1")
        self._source = source
        self._service = service
        self._max_concurrency = max_concurrency
        self._clock = clock

    async def collect(self, experiment_ids: Sequence[int]) -> CollectionReport:
        semaphore = asyncio.Semaphore(self._max_concurrency)

        async def fetch_one(experiment_id: int) -> list[float]:
            async with semaphore:
                return await self._source.fetch(experiment_id)

        outcomes = await asyncio.gather(*(fetch_one(i) for i in experiment_ids),
```

```
return_exceptions=True)
        report = CollectionReport()
        for experiment_id, outcome in zip(experiment_ids, outcomes, strict=True):
            if isinstance(outcome, BaseException):
                report.failed[experiment_id] = f"{type(outcome).__name__}: {outcome}"
                continue
            try:
                for value in outcome:
                    self._service.add_result(experiment_id, value, self._clock())
                    report.collected += 1
            except ExperimentError as error:
                report.failed[experiment_id] = f"{type(error).__name__}: {error}"
        return report
```

4. Хід виконання

4.1. Структура production code і test suite

Лістинг 6.2. Production code (src/experiment_registry)

```
experiment_registry/
├── commands/
│   ├── __init__.py
│   ├── analyze.py
│   ├── demo.py
│   ├── domain.py
│   ├── importer.py
│   └── stream.py
├── domain/
│   ├── __init__.py
│   ├── dto.py
│   ├── exporters.py
│   ├── models.py
│   ├── processing.py
│   ├── protocols.py
│   ├── repositories.py
│   ├── services.py
│   ├── sources.py
│   ├── strategies.py
│   └── value_objects.py
├── importer/
│   ├── __init__.py
│   ├── config.py
│   ├── exporters.py
│   ├── logging_config.py
│   ├── readers.py
│   ├── services.py
│   └── validators.py
├── streaming/
│   ├── __init__.py
│   ├── analytics.py
│   ├── datagen.py
│   ├── eager.py
│   ├── filters.py
│   ├── iterators.py
│   ├── parsers.py
│   ├── pipeline.py
│   └── readers.py
├── __init__.py
├── analytics.py
├── config.py
├── data.py
├── decorators.py
├── exceptions.py
├── main.py
├── models.py
├── processors.py
└── services.py
```

Лістинг 6.3. Test suite і код експериментів

```
tests/
├── integration/
```

```
│   ├── test_application_flow.py
│   ├── test_experiment_pipeline.py
│   └── test_import_pipeline.py
├── unit/
│   ├── test_async_source.py
│   ├── test_configuration.py
│   ├── test_core_services.py
│   ├── test_exporters.py
│   ├── test_models.py
│   ├── test_readers.py
│   ├── test_repositories.py
│   ├── test_services.py
│   ├── test_strategies.py
│   ├── test_streaming.py
│   ├── test_validators.py
│   └── test_value_objects.py
└── conftest.py

experiments/
├── buggy_metrics.py
├── classification.py
├── error_metrics.py
├── test_branch_full.py
├── test_branch_partial.py
├── test_coverage_strong.py
├── test_coverage_weak.py
├── test_flaky_bad.py
├── test_flaky_fixed.py
├── test_mock_vs_fake.py
├── test_tmp_isolation.py
└── test_weak_vs_strong.py
```

Тести розділені на `unit/` (окремі класи й функції, без файлів і мережі, крім tmp_path) та `integration/` (кілька компонентів разом, позначені маркером `integration`). Налаштування pytest і coverage зберігаються в `pyproject.toml`.

Лістинг 6.4. Налаштування pytest і coverage

```toml
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = ["-ra", "--strict-markers"]
markers = ["integration: integration tests (several components together)"]
asyncio_default_fixture_loop_scope = "function"

[tool.coverage.run]
branch = true
source = ["experiment_registry"]

[tool.coverage.report]
show_missing = true
skip_covered = false
fail_under = 80
```

4.2. Test matrix

Таблиця 6.1. Мінімальна матриця тестування

Категорія	Приклади тестів
Normal	валідний запис → payload з одиницею; звіт із метриками; tolerant-імпорт
Minimum boundary	result = 0 і 0.0; межі `ResultRange` 100.0 включно; дата = сьогодні
Maximum boundary	result = 1000; 400.0 у діапазоні, 400.001 — ні; batch size > len
Empty	порожня `ExperimentSeries`; `analyze([])`; експеримент без результатів не публікується
Invalid type/value	`x1`, `0` як ID; `wrong`, `nan`, `inf`; `2026-02-30`; невідомий параметр
Missing data	відсутні колонка, config.yaml, input-файл, каталог output, змінна середовища
Dependency failure	exporter `side_effect=ExportError`; async source `TimeoutError`

Категорія	Приклади тестів
Repeated operation	повторний ID результату чи експерименту; повторне додавання в серію
State transition	strict → abort без output; tolerant → output + rejected.csv

4.3. Fixtures

Спільні fixtures оголошені в `conftest.py` і автоматично доступні всім тестам. `experiment` будується з fixture `researcher` (залежність між fixtures), `make_measurement` — fixture factory, `config_file` створює YAML у тимчасовому каталозі. Fixture `importer_logger` використовує `yield`: код після `yield` виконується як teardown і закриває файлові handlers логера навіть після невдалого тесту.

Лістинг 6.5. Fixtures у conftest.py

```
@pytest.fixture
def experiment(researcher: Researcher) -> Experiment:
    """Experiment with three temperature results."""
    item = Experiment(1, "Thermal conductivity of Al alloy", researcher, "temperature", "K")
    for result_id, value in enumerate((293.4, 297.1, 301.8), start=1):
        item.add_result(ExperimentResult(result_id, Measurement("temperature", value, "K"),
                                         date(2026, 2, result_id)))
    return item


@pytest.fixture
def make_measurement() -> Callable[..., Measurement]:
    """Fixture factory: builds measurements with sensible defaults."""

    def factory(value: float = 300.0, parameter: str = "temperature", unit: str = "K") ->
Measurement:
        return Measurement(parameter, value, unit)

    return factory


@pytest.fixture
def config_file(tmp_path: Path, config_data: dict[str, object]) -> Path:
    path = tmp_path / "config.yaml"
    path.write_text(yaml.safe_dump(config_data), encoding="utf-8")
    return path


@pytest.fixture
def importer_logger() -> Iterator[logging.Logger]:
    """Fixture with teardown: closes file handlers created by setup_logging()."""
    logger = logging.getLogger("experiment_registry.importer")
    yield logger
    for handler in list(logger.handlers):
        logger.removeHandler(handler)
        handler.close()
```

4.4. Приклади unit tests, parametrization і boundary tests

Тести мають назви, що описують очікувану поведінку, і структуру Arrange – Act – Assert. Parametrization дозволяє перевірити багато вхідних даних однією функцією: кожен набір параметрів — окремий тест зі своїм id у звіті.

Лістинг 6.6. Parametrization і boundary test для value objects

```
@pytest.mark.parametrize(
    ("parameter", "value", "unit"),
    [("", 1.0, "K"), ("temperature", 1.0, " "), ("temperature", float("nan"), "K"),
     ("temperature", float("inf"), "K")],
    ids=["empty-parameter", "empty-unit", "nan", "infinity"],
```

```
)
def test_invalid_measurement_raises_validation_error(parameter: str, value: float, unit: str) ->
None:
    with pytest.raises(ExperimentValidationError):
        Measurement(parameter, value, unit)


@pytest.mark.parametrize(
    ("value", "expected"),
    [(99.999, False), (100.0, True), (250.0, True), (400.0, True), (400.001, False)],
)
def test_result_range_includes_boundaries(value: float, expected: bool) -> None:
    assert (value in ResultRange(100, 400)) is expected
```

Лістинг 6.7. Невалідні та граничні значення при імпорті

```
@pytest.mark.parametrize(
    ("field", "value"),
    [
        ("experiment_id", "x1"), ("experiment_id", "0"), ("experiment_id", ""),
        ("name", "  "), ("researcher", ""),
        ("parameter", "colour"),
        ("result", "wrong"), ("result", "nan"), ("result", "-0.1"), ("result", "1000.01"),
        ("date", "2026-02-30"), ("date", "03.02.2026"), ("date", "2026-11-09"),
    ],
)
def test_invalid_field_raises_record_validation_error(field: str, value: str) -> None:
    with pytest.raises(RecordValidationError) as exc_info:
        validate_record(make_row(**{field: value}), line=7, rules=RULES, today=TODAY)

    assert exc_info.value.field == field
    assert exc_info.value.line == 7


@pytest.mark.parametrize("value", ["0", "0.0", "1000", "1000.0"])
def test_boundary_results_are_accepted(value: str) -> None:
    assert validate_record(make_row(result=value), line=2, rules=RULES, today=TODAY)["result"]
== float(value)
```

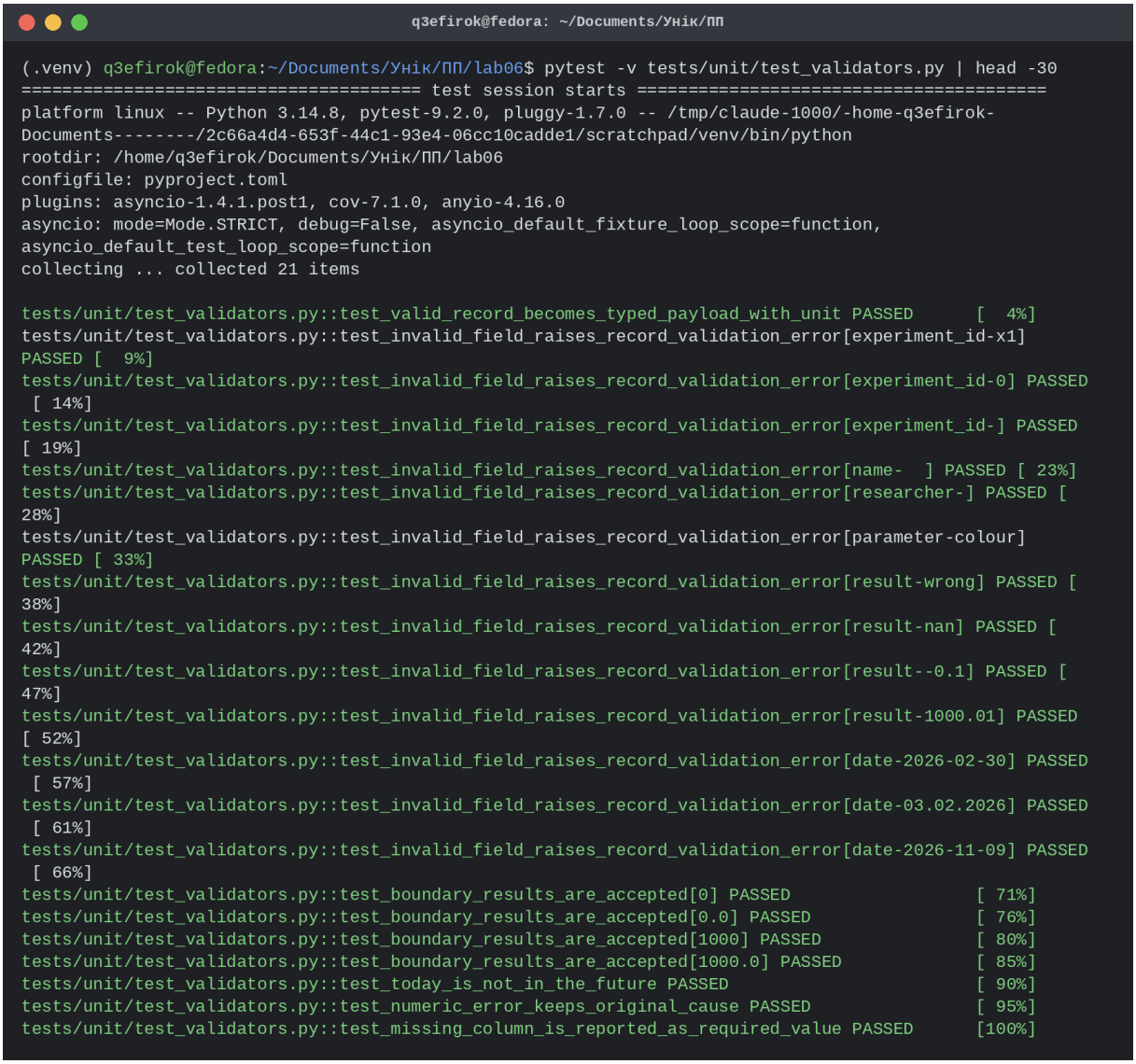


Рис. 6.1. Згенеровані параметризовані тести (pytest -v)

4.5. pytest.raises, custom exception і pytest.approx

`pytest.raises` перевіряє тип винятку, а аргумент `match` — текст повідомлення; через `exc_info` доступні атрибути власного винятку (`field`, `line`) і причина в `__cause__`. Для дійсних чисел використано `pytest.approx`, оскільки середнє та стандартне відхилення мають похибку округлення.

Лістинг 6.8. Перевірка custom exception і exception chaining

```
def test_numeric_error_keeps_original_cause() -> None:
    with pytest.raises(RecordValidationError) as exc_info:
        validate_record(make_row(result="wrong"), line=3, rules=RULES, today=TODAY)

    assert isinstance(exc_info.value.__cause__, ValueError)
    assert "line 3: result='wrong'" in str(exc_info.value)
```

Лістинг 6.9. pytest.approx для метрик

```
@pytest.mark.parametrize(
    ("strategy", "expected"),
    [(MeanStrategy(), 297.433333), (MinimumStrategy(), 293.4), (MaximumStrategy(), 301.8),
     (StandardDeviationStrategy(), 4.209909)],
    ids=lambda item: getattr(item, "name", str(item)),
```

```
)
def test_strategy_calculates_metric(strategy: AnalysisStrategy, expected: float) -> None:
    assert strategy.analyze(VALUES) == pytest.approx(expected, rel=1e-6)
```

4.6. tmp_path і monkeypatch

Файлові тести працюють у `tmp_path` — унікальному тимчасовому каталозі для кожного тесту, тож вони не залежать один від одного й не залишають файлів у репозиторії. `monkeypatch` тимчасово змінює змінні середовища (`setenv`, `delenv`) та атрибути (`builtins.input` для інтерактивного меню) і автоматично повертає їх після тесту.

Лістинг 6.10. tmp_path: атомарний експорт не пошкоджує попередній файл

```
def test_failed_serialization_keeps_previous_output(tmp_path: Path) -> None:
    target = export_json(tmp_path / "out.json", {"version": 1})

    with pytest.raises(ExportError) as exc_info:
        export_json(target, {"version": 2, "bad": object()})

    assert isinstance(exc_info.value.__cause__, TypeError)
    assert json.loads(target.read_text(encoding="utf-8")) == {"version": 1}
    assert list(tmp_path.glob("*.tmp")) == []
```

Лістинг 6.11. monkeypatch.setenv

```
def test_environment_variable_overrides_yaml(config_file: Path, monkeypatch:
pytest.MonkeyPatch) -> None:
    monkeypatch.setenv("EXPERIMENTS_SKIP_INVALID", "false")
    monkeypatch.setenv("EXPERIMENTS_LOG_LEVEL", "debug")

    config = load_config(config_file)

    assert config.processing.skip_invalid is False
    assert config.logging.level == "DEBUG"
```

Лістинг 6.12. monkeypatch для builtins.input

```
def test_interactive_menu_with_scripted_input(monkeypatch: pytest.MonkeyPatch,
                                              capsys: pytest.CaptureFixture[str]) -> None:
    answers = iter(["4", "temperature", "5", "100", "50", "2", "Probe", "Andrii Bondar",
"density",
                    "abc", "12.5", "2026-03-15", "3", "Andrii Bondar", "7", "0"])
    monkeypatch.setattr("builtins.input", lambda prompt="": next(answers))
    experiments = create_demo_experiments()

    run_menu(experiments)

    out = capsys.readouterr().out
    assert "count=4 average=255.32" in out
    assert "Invalid range: 100.0 > 50.0" in out
    assert "Please enter a number." in out
    assert len(experiments) == 9 and experiments[-1].parameter == "density"
    assert "Unknown command." in out and "Goodbye." in out
```

4.7. Mock, return_value, side_effect та interaction assertions

`Mock(spec=ReportExporter)` створює test double з тими самими методами, що й протокол, тому звернення до неіснуючого методу одразу дає помилку. `return_value` задає відповідь залежності, `side_effect` — виняток. Interaction assertions (`assert_called_once_with`, `assert_not_called`) перевіряють не стан,

а те, як сервіс взаємодіяв із залежністю: у failure path важливо, що після помилки exporter нічого не записано в sink.

Лістинг 6.13. Mock з return_value, side_effect і перевіркою взаємодії

```
def test_publish_sends_exported_text_to_sink(experiment: Experiment) -> None:
    exporter = Mock(spec=ReportExporter)
    exporter.export.return_value = "REPORT #1"
    sink = Mock(spec=ReportSink)

    written = ReportingService([MeanStrategy()], exporter, sink).publish([experiment])

    assert written == 1
    exporter.export.assert_called_once()
    sink.write.assert_called_once_with("REPORT #1")


def test_exporter_failure_does_not_write_to_sink(experiment: Experiment) -> None:
    """Failure path: the error propagates and no partial report reaches the sink."""
    exporter = Mock(spec=ReportExporter)
    exporter.export.side_effect = ExportError("disk is full")
    sink = Mock(spec=ReportSink)

    with pytest.raises(ExportError, match="disk is full"):
        ReportingService([MeanStrategy()], exporter, sink).publish([experiment])
    sink.write.assert_not_called()
```

4.8. AsyncMock

Асинхронне джерело вимірювань замінено на `AsyncMock`: виклик `fetch()` повертає корутину, результат якої задається `return_value` або `side_effect`. Перевіряються `assert_awaited_once_with` і `await_count`, обробка `TimeoutError` одного джерела без зупинки інших та обмеження паралельності семафором. Асинхронні тести запускає плагін pytest-asyncio (`@pytest.mark.asyncio`).

Лістинг 6.14. AsyncMock: успішний сценарій і failure path

```
@pytest.mark.asyncio
async def test_pipeline_records_values_from_async_source(experiment_service: ExperimentService)
-> None:
    source = AsyncMock()
    source.fetch.return_value = [293.4, 297.1]
    pipeline = ExperimentPipeline(source, experiment_service, clock=lambda: FIXED_DAY)

    report = await pipeline.collect([1])

    source.fetch.assert_awaited_once_with(1)
    assert report.collected == 2 and report.failed == {}
    assert experiment_service.find_by_researcher("Olena Kovalenko")[0].values() == [293.4,
297.1]


@pytest.mark.asyncio
async def test_source_failure_is_reported_and_other_experiments_continue(
    experiment_service: ExperimentService,
) -> None:
    async def fetch(experiment_id: int) -> list[float]:
        if experiment_id == 2:
            raise TimeoutError("device timeout")
        return [300.0]

    source = AsyncMock()
    source.fetch.side_effect = fetch

    report = await ExperimentPipeline(source, experiment_service, clock=lambda:
FIXED_DAY).collect([1, 2])

    assert report.collected == 1
    assert report.failed == {2: "TimeoutError: device timeout"}
```

```
    assert source.fetch.await_count == 2
```

4.9. Integration tests

Integration tests перевіряють кілька реальних компонентів разом без mocks: конфігурацію, читання CSV, валідацію, доменну модель, JSON-експорт, `rejected.csv` і журнал; асинхронне джерело → сервіс → стратегії → файл звіту; консольний застосунок (усі підкоманди та коди завершення 0/1/2).

Лістинг 6.15. Integration test імпорту

```
def test_tolerant_import_exports_valid_results_and_rejected_rows(
    config_file: Path, importer_logger: logging.Logger,
) -> None:
    config = load_config(config_file, env={})
    service = ImportService(config, setup_logging(config.logging, console=False))

    stats = service.run()

    document = json.loads(config.output_path.read_text(encoding="utf-8"))
    assert (stats.processed, stats.valid, stats.invalid, stats.exported) == (4, 3, 1, 3)
    assert [r["result"] for r in document["results"]] == [293.4, 301.8, 101.3]
    assert document["experiments"][0]["metrics"]["average"] == pytest.approx(297.6)
    assert config.errors_path is not None
    assert "result,wrong" in config.errors_path.read_text(encoding="utf-8")
    assert "Skipped invalid record: line 5" in config.logging.file.read_text(encoding="utf-8")
```

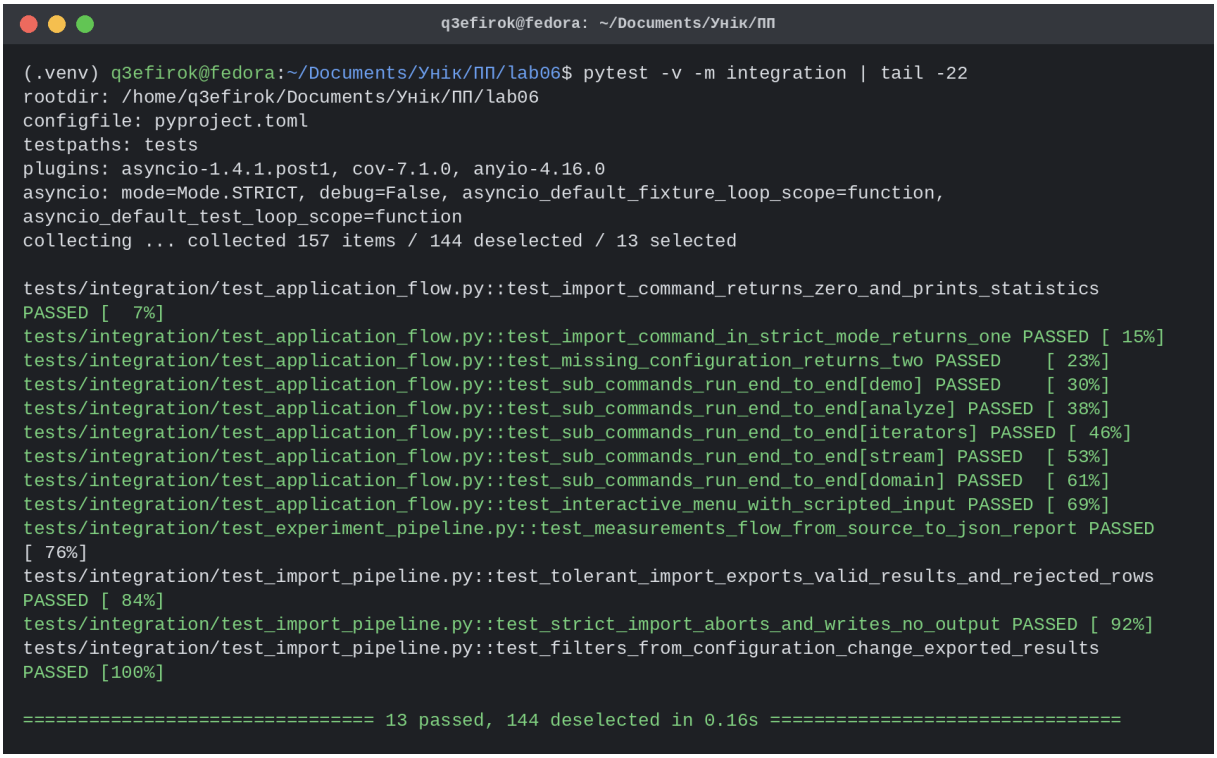


Рис. 6.2. Integration tests (pytest -m integration)

4.10. Результат запуску tests



Рис. 6.3. Запуск усього test suite



Рис. 6.4. Лише unit tests

Таблиця 6.2. Підсумок test run

Показник	Значення
Усього тестів (з урахуванням параметризації)	157
Passed	157
Failed	0
Statement coverage + branch coverage (TOTAL)	92%
Statements / missed	1648 / 112
Branches / partial	348 / 47

4.11. Coverage report і branch coverage

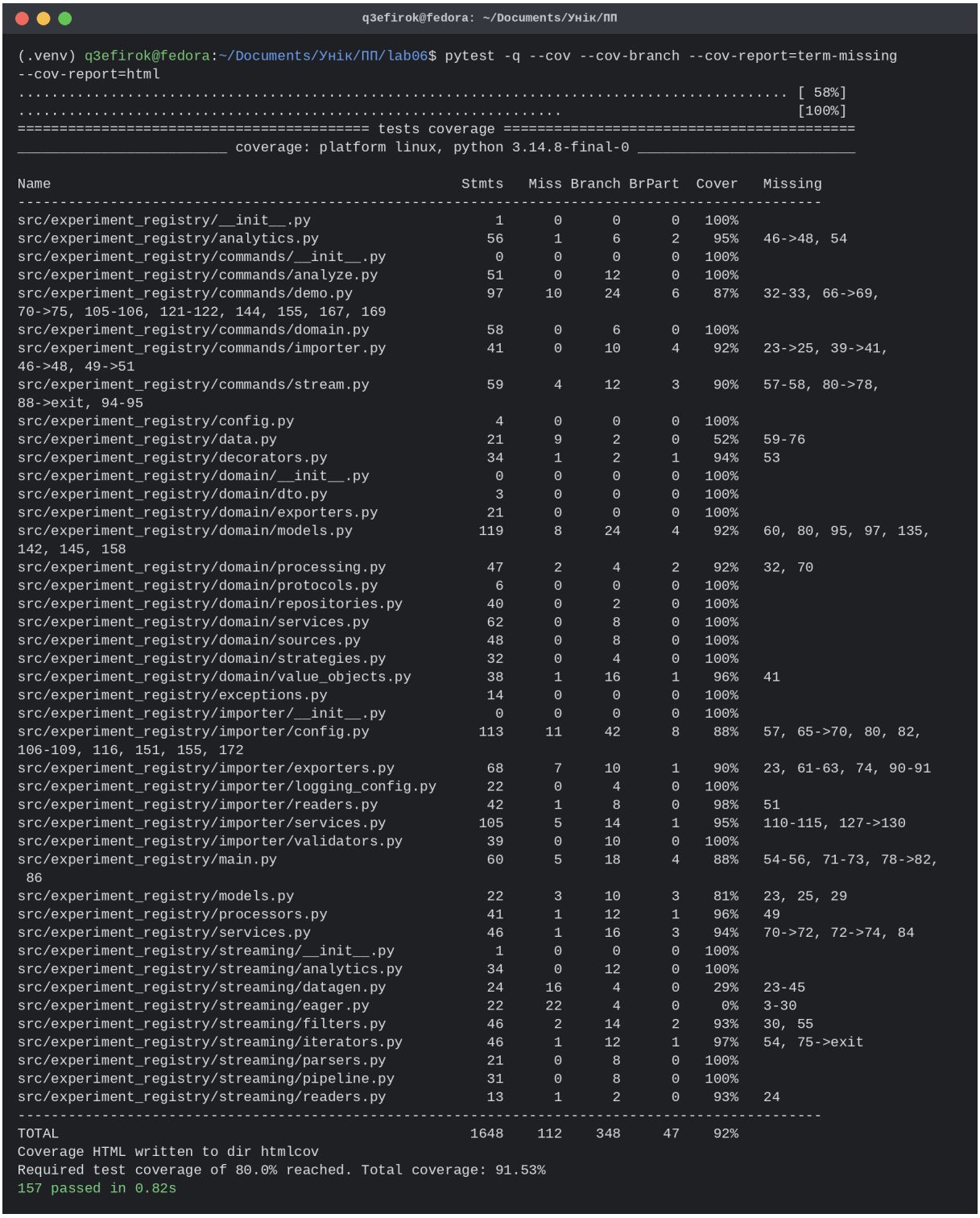
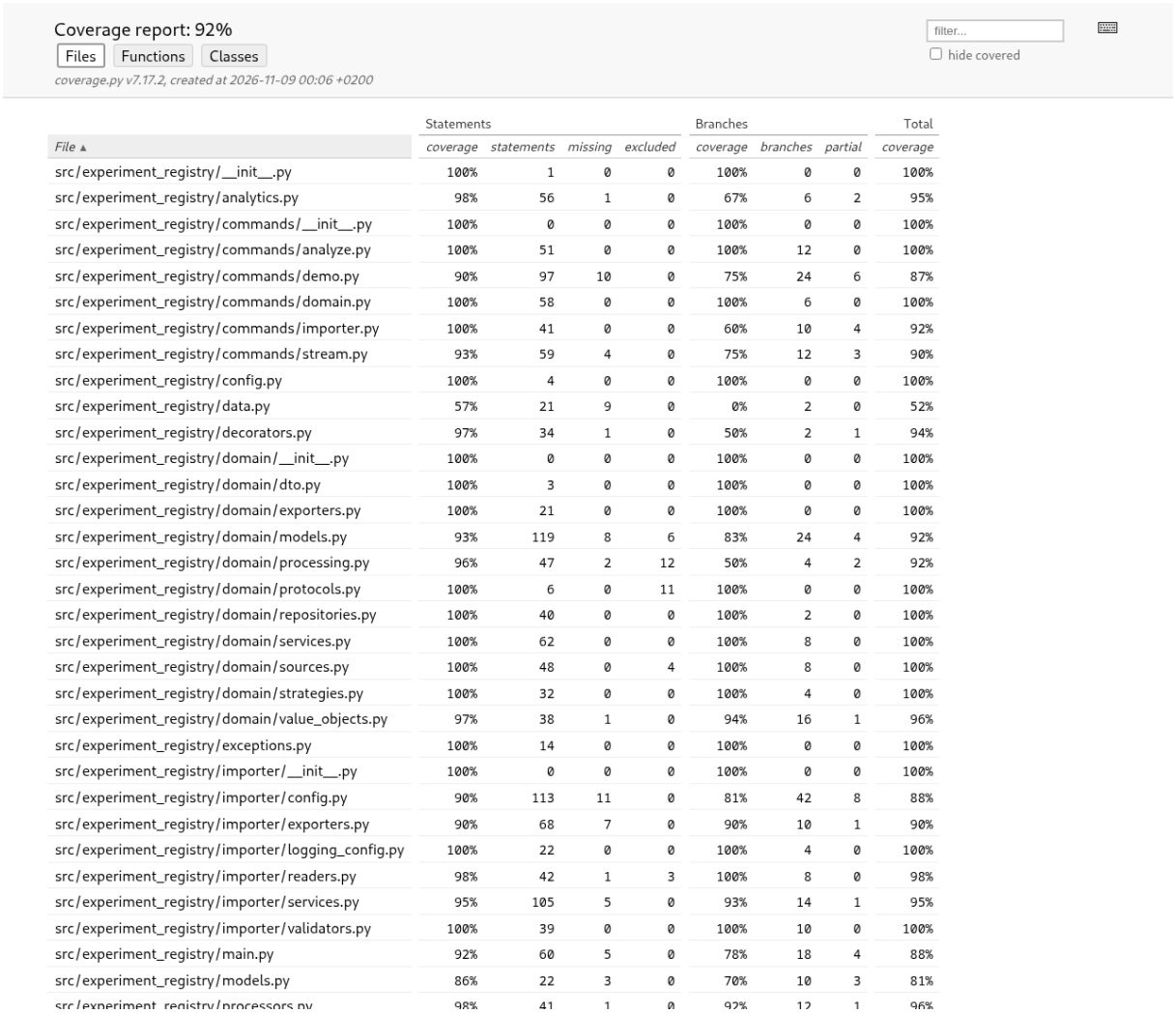


Рис. 6.5. Coverage report з branch coverage



Coverage report: 92%

Files | Functions | Classes

filter...

hide covered

coverage.py v7.17.2, created at 2026-11-09 00:06 +0200

	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src/experiment_registry/__init__.py	100%	1	0	0	100%	0	0	100%
src/experiment_registry/analytics.py	98%	56	1	0	67%	6	2	95%
src/experiment_registry/commands/__init__.py	100%	0	0	0	100%	0	0	100%
src/experiment_registry/commands/analyze.py	100%	51	0	0	100%	12	0	100%
src/experiment_registry/commands/demo.py	90%	97	10	0	75%	24	6	87%
src/experiment_registry/commands/domain.py	100%	58	0	0	100%	6	0	100%
src/experiment_registry/commands/importer.py	100%	41	0	0	60%	10	4	92%
src/experiment_registry/commands/stream.py	93%	59	4	0	75%	12	3	90%
src/experiment_registry/config.py	100%	4	0	0	100%	0	0	100%
src/experiment_registry/data.py	57%	21	9	0	0%	2	0	52%
src/experiment_registry/decorators.py	97%	34	1	0	50%	2	1	94%
src/experiment_registry/domain/__init__.py	100%	0	0	0	100%	0	0	100%
src/experiment_registry/domain/dto.py	100%	3	0	0	100%	0	0	100%
src/experiment_registry/domain/exporters.py	100%	21	0	0	100%	0	0	100%
src/experiment_registry/domain/models.py	93%	119	8	6	83%	24	4	92%
src/experiment_registry/domain/processing.py	96%	47	2	12	50%	4	2	92%
src/experiment_registry/domain/protocols.py	100%	6	0	11	100%	0	0	100%
src/experiment_registry/domain/repositories.py	100%	40	0	0	100%	2	0	100%
src/experiment_registry/domain/services.py	100%	62	0	0	100%	8	0	100%
src/experiment_registry/domain/sources.py	100%	48	0	4	100%	8	0	100%
src/experiment_registry/domain/strategies.py	100%	32	0	0	100%	4	0	100%
src/experiment_registry/domain/value_objects.py	97%	38	1	0	94%	16	1	96%
src/experiment_registry/exceptions.py	100%	14	0	0	100%	0	0	100%
src/experiment_registry/importer/__init__.py	100%	0	0	0	100%	0	0	100%
src/experiment_registry/importer/config.py	90%	113	11	0	81%	42	8	88%
src/experiment_registry/importer/exporters.py	90%	68	7	0	90%	10	1	90%
src/experiment_registry/importer/logging_config.py	100%	22	0	0	100%	4	0	100%
src/experiment_registry/importer/readers.py	98%	42	1	3	100%	8	0	98%
src/experiment_registry/importer/services.py	95%	105	5	0	93%	14	1	95%
src/experiment_registry/importer/validators.py	100%	39	0	0	100%	10	0	100%
src/experiment_registry/main.py	92%	60	5	0	78%	18	4	88%
src/experiment_registry/models.py	86%	22	3	0	70%	10	3	81%
src/experiment_registry/processors.py	98%	41	1	0	92%	12	1	96%

Рис. 6.6. HTML coverage report (htmlcov/index.html)

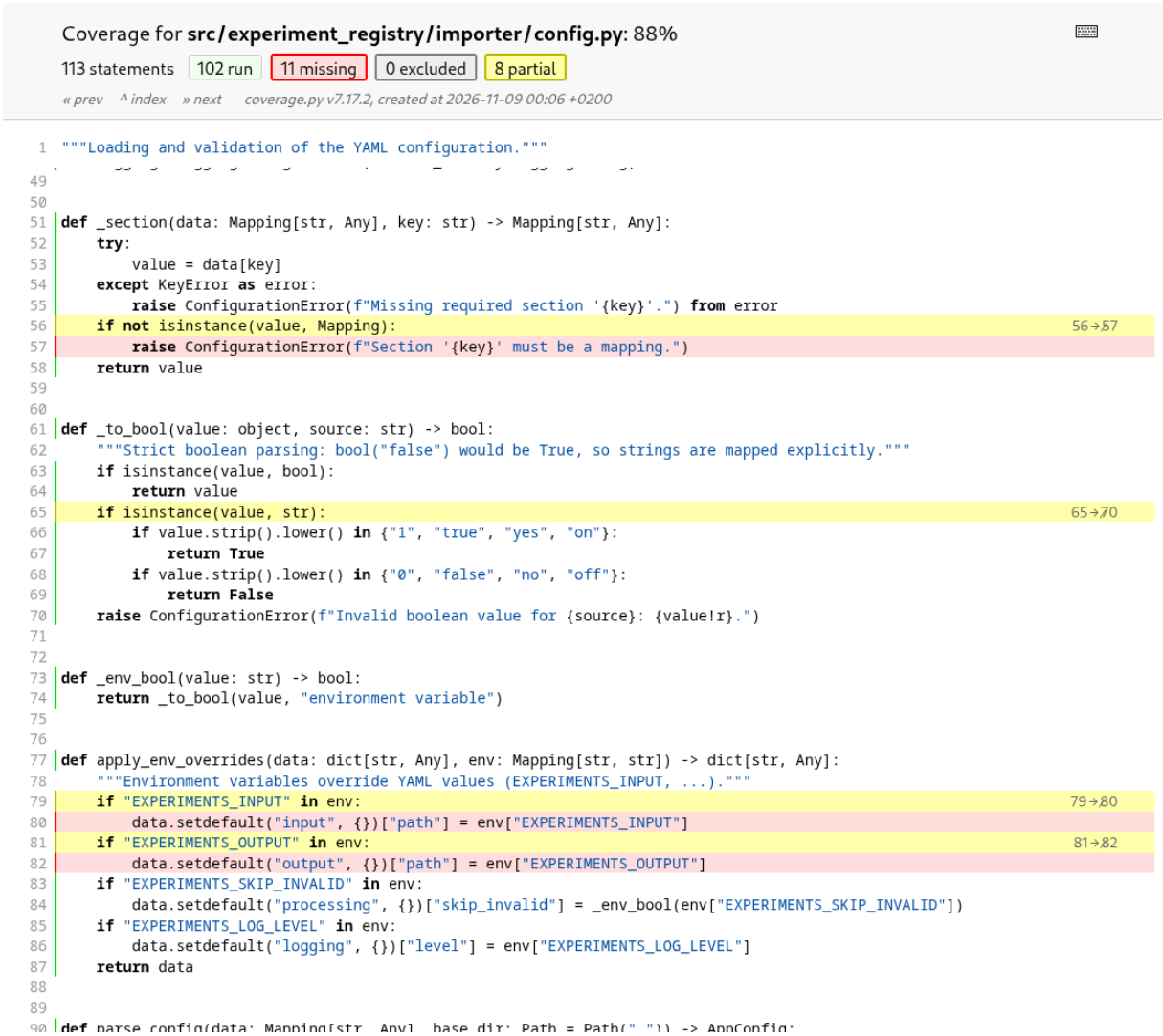


Рис. 6.7. HTML coverage report модуля importer/config.py: червоним позначено невиконані рядки, жовтим — частково виконані розгалуження

4.12. Аналіз недостатньо протестованих paths

Таблиця 6.3. Модулі з найнижчим покриттям

Модуль	Stmts	Miss	Branch	BrPart	Cover
`streaming/eager.py`	22	22	4	0	0%
`streaming/datagen.py`	24	16	4	0	29%
`data.py`	21	9	2	0	52%
`importer/readers.py`	42	1	8	0	98%
`importer/config.py`	113	11	42	8	88%
`commands/demo.py`	97	10	24	6	87%

- `streaming/eager.py`, `streaming/datagen.py` і `generate_experiments` у `data.py` використовуються лише benchmark-скриптами лабораторних №2–3; вони не впливають на доменну логіку, тому їх свідомо не покрито тестами.

- `importer/config.py` (рядки 57, 80, 82, 106–109, 116, 151, 155, 172): не перевірено секцію, що не є словником, перевизначення `EXPERIMENTS_INPUT`/`EXPERIMENTS_OUTPUT`, відсутній ключ усередині секції, `result_range` не у вигляді списку, `low > high`, від'ємний `indent` і корінь YAML, що не є словником. Це гілки обробки помилок конфігурації — кандидати на розширення параметризованого тесту `test_invalid_configuration_is_rejected`.
- `importer/readers.py` (рядок 51): не виконується гілка `OSError` для JSON Lines; її можна перевірити, передавши через `tmp_path` каталог замість файлу.
- `commands/demo.py`: інтерактивні гілки повторного введення дати та статистики для порожнього набору — їх можна покрити ще одним сценарієм monkeypatch для `input`.

Високий відсоток покриття сам по собі не гарантує якості: smoke-тести підкоманд CLI виконують сотні рядків, але перевіряють лише ключові фрагменти виводу. Тому важливіше, що доменна логіка, валідація і сервіси покриті тестами з конкретними assertions.

4.13. Знайдені тестами дефекти

Під час написання тестів для ще не покритих гілок (JSON Lines reader і булеві значення конфігурації) чотири нові тести впали і виявили два реальні дефекти production code:

Лістинг 6.16. Результат запуску нових тестів до виправлення

```
.....F..................FF..F                                    [100%]
================================== FAILURES ===================================
E   Failed: DID NOT RAISE RecordValidationError
tests/unit/test_readers.py:62: Failed: DID NOT RAISE RecordValidationError
E   AssertionError: assert True is False
     +  where True = ProcessingConfig(skip_invalid=True, researcher=None,
result_range=None).skip_invalid
     +    where ProcessingConfig(skip_invalid=True, researcher=None, result_range=None) =
AppConfig(input_path=PosixPath('/tmp/pytest-of-q3efirok/pytest-2/test_skip_invalid_accepts_yaml1/
experiments.csv'), ou...mp/pytest-of-q3efirok/pytest-2/test_skip_invalid_accepts_yaml1/logs/
importer.log'), max_bytes=1000000, backup_count=3)).processing
tests/unit/test_configuration.py:90: AssertionError: assert True is False
E   AssertionError: assert True is False
     +  where True = ProcessingConfig(skip_invalid=True, researcher=None,
result_range=None).skip_invalid
     +    where ProcessingConfig(skip_invalid=True, researcher=None, result_range=None) =
AppConfig(input_path=PosixPath('/tmp/pytest-of-q3efirok/pytest-2/test_skip_invalid_accepts_yaml2/
experiments.csv'), ou...mp/pytest-of-q3efirok/pytest-2/test_skip_invalid_accepts_yaml2/logs/
importer.log'), max_bytes=1000000, backup_count=3)).processing
tests/unit/test_configuration.py:90: AssertionError: assert True is False
E   Failed: DID NOT RAISE ConfigurationError
tests/unit/test_configuration.py:96: Failed: DID NOT RAISE ConfigurationError
=========================== short test summary info ============================
FAILED tests/unit/test_readers.py::test_json_null_value_is_treated_as_missing
FAILED tests/unit/test_configuration.py::test_skip_invalid_accepts_yaml_booleans_and_strings[false-
False]
FAILED tests/unit/test_configuration.py::test_skip_invalid_accepts_yaml_booleans_and_strings[no-False]
FAILED tests/unit/test_configuration.py::test_unrecognized_boolean_in_yaml_is_rejected
4 failed, 25 passed in 0.07s
```

1. **Рядок "false" у YAML вмикав tolerant-режим.** `bool("false")` у Python дорівнює `True`, тому `skip_invalid: "false"` (у лапках) або `"no"` мовчки вимикав strict-режим, а довільний рядок `create_dirs: "sometimes"` приймався без

помилки. Виправлено функцією `_to_bool`, яка явно розпізнає булеві рядки й інакше піднімає `ConfigurationError`.

2. **`null` у JSON Lines проходив валідацію.** Reader перетворював значення через `str()`, тому `null` ставав рядком `"None"`, і запис без дослідника імпортувався з researcher «None». Виправлено: `None` перетворюється на порожній рядок, який validator відхиляє.

Лістинг 6.17. Виправлення в readers.py (git show)

```
                            data = json.loads(line)
                        except json.JSONDecodeError as error:
                            raise DataImportError(f"{path}:{number}: invalid JSON") from error
-                       yield number, {key: str(value) for key, value in data.items()}
+                       yield number, {key: "" if value is None else str(value) for key, value
in data.items()}
            except OSError as error:
                raise DataImportError(f"{path}: cannot read file ({error.strerror})") from error
```

Також регресійним тестом `test_add_result_rejects_other_parameter` закріплено виправлення дефекту, знайденого в лабораторній №5 (запис pressure потрапляв в експеримент voltage).

4.14. Статична перевірка

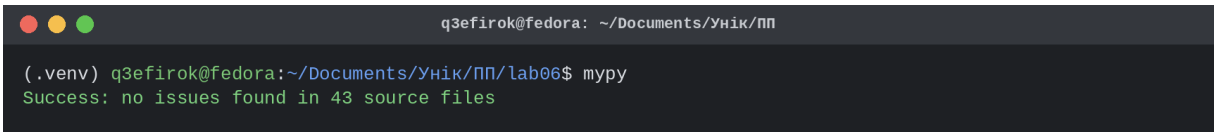


Рис. 6.8. mypy після додавання асинхронного конвеєра

5. Експериментальна частина

5.1. Експеримент 1. Weak vs strong tests

Лістинг 6.18. Слабкий і сильний тест для коректної та дефектної стратегії

```
VALUES = [293.4, 297.1, 301.8]


@pytest.mark.parametrize("strategy", [MeanStrategy(), BuggyMeanStrategy()],
ids=["correct", "buggy"])
def test_mean_weak(strategy: AnalysisStrategy) -> None:
    strategy.analyze(VALUES)  # no assertion: passes for any result


@pytest.mark.parametrize("strategy", [MeanStrategy(), BuggyMeanStrategy()],
ids=["correct", "buggy"])
def test_mean_strong(strategy: AnalysisStrategy) -> None:
    assert strategy.analyze(VALUES) == pytest.approx(297.433333)
```

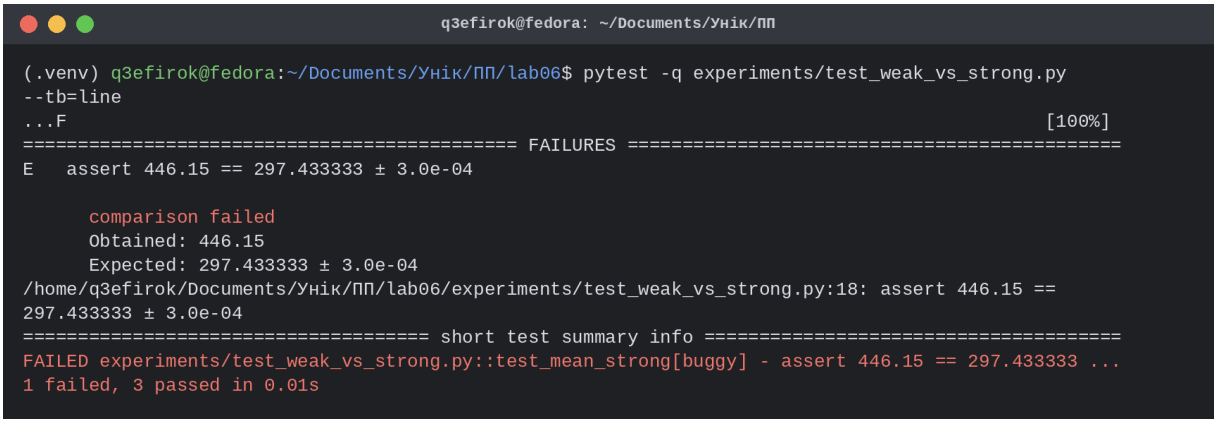


Рис. 6.9. Слабкий тест проходить навіть для дефектної реалізації

Слабкий тест лише викликає код і проходить для обох реалізацій, хоча `BuggyMeanStrategy` ділить на n − 1. Сильний тест з `pytest.approx` виявив дефект. Висновок: execution ≠ verification.

5.2. Експеримент 2. Coverage без assertions

Лістинг 6.19. Функція з навмисною помилкою

```
"""Deliberately defective function for the coverage experiment (not part of the
package)."""


def relative_error(measured: float, expected: float) -> float:
    """Defect: divides by the measured value and loses the absolute value."""
    return (measured - expected) / measured
```

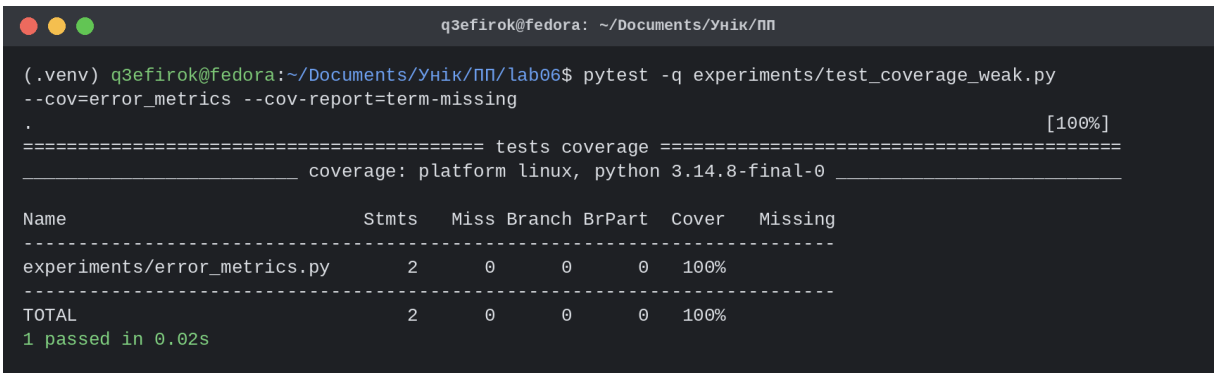


Рис. 6.10. 100% coverage тестом без assertions

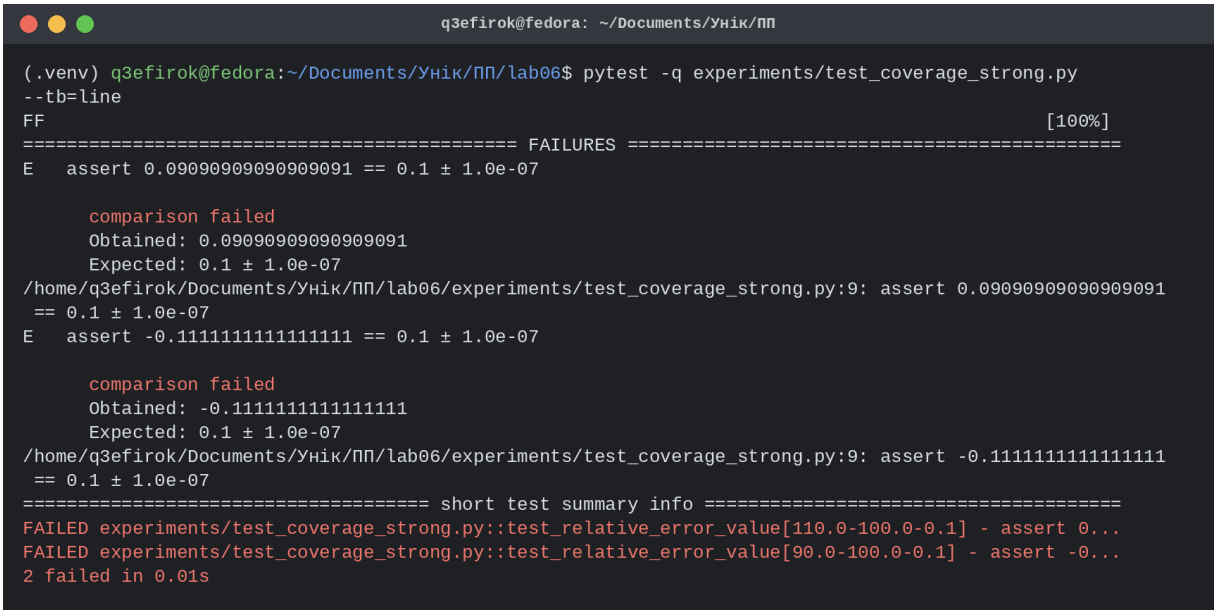


Рис. 6.11. Тест зі змістовними assertions виявляє помилку

Coverage показує, що код виконувався, але не гарантує, що результат перевірено: тест без assertions дав 100% покриття функції з двома помилками (ділення на виміряне значення та відсутність `abs`). Лише перевірка конкретних значень показала дефект.

5.3. Експеримент 3. Branch coverage

Лістинг 6.20. Функція з трьома гілками

```
"""Function with three branches for the branch-coverage experiment."""


def classify_accuracy(relative_error: float) -> str:
    if relative_error <= 0.01:
        return "high"
    if relative_error <= 0.05:
        return "acceptable"
    return "low"
```

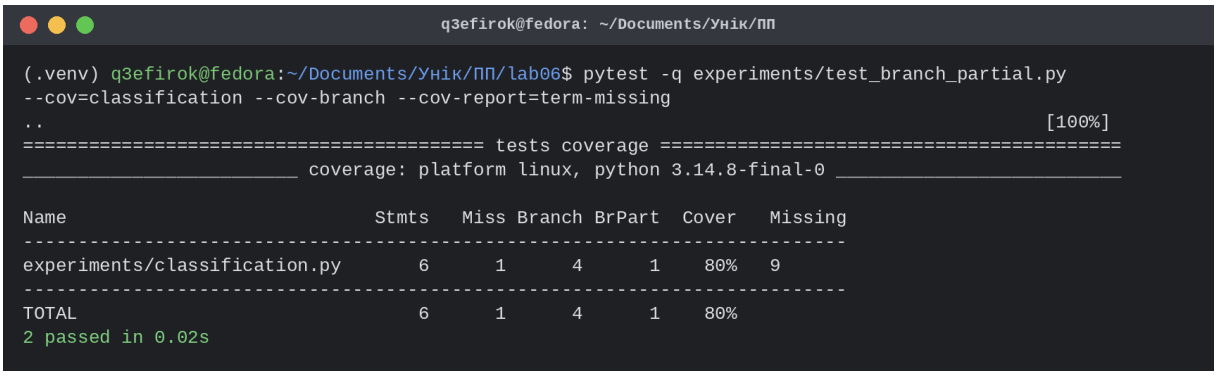


Рис. 6.12. Тести лише для 0.005 і 0.03

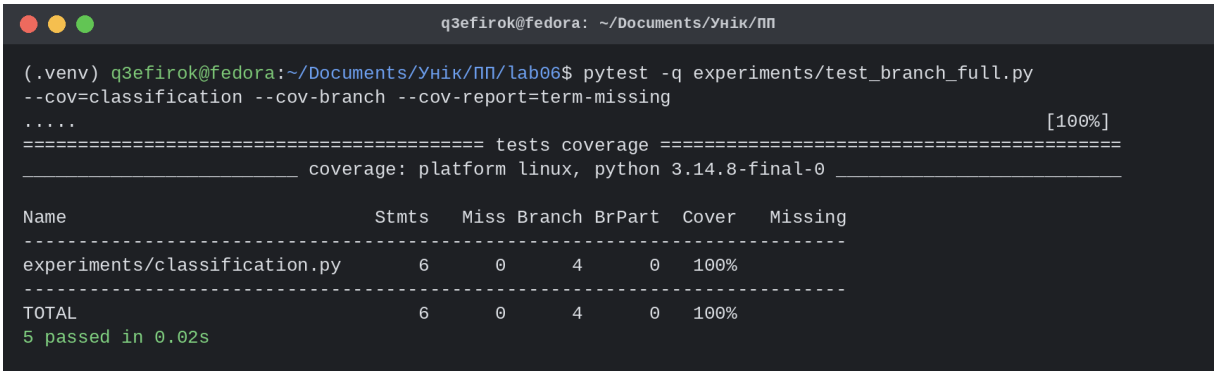


Рис. 6.13. Додано 0.1 та граничні значення 0.01 і 0.05

З двома тестами statement coverage функції 83% (не виконано один рядок), але branch coverage показує й частково виконане розгалуження (BrPart = 1): умову `<= 0.05` жодного разу не було хибною. Після додавання 0.1 і граничних значень покрито 100% рядків і гілок.

5.4. Експеримент 4. Mock vs fake

Лістинг 6.21. Той самий сценарій з Mock і з fake-репозиторієм

```
def test_find_by_researcher_with_mock() -> None:
    repository = Mock(spec=ExperimentRepository)
    repository.find_by_researcher.return_value = ["experiment #1"]
    service = ExperimentService(repository, Mock(spec=ResearcherRepository))

    assert service.find_by_researcher("Olena Kovalenko") == ["experiment #1"]
    repository.find_by_researcher.assert_called_once_with("Olena Kovalenko")  #
interaction


def test_find_by_researcher_with_fake() -> None:
    repository = ExperimentRepository()  # real in-memory implementation used as a fake
    repository.add(Experiment(1, "Thermal conductivity", OLENA, "temperature", "K"))
    service = ExperimentService(repository, ResearcherRepository())

    found = service.find_by_researcher("olena kovalenko")

    assert [item.id for item in found] == [1]  # state
```

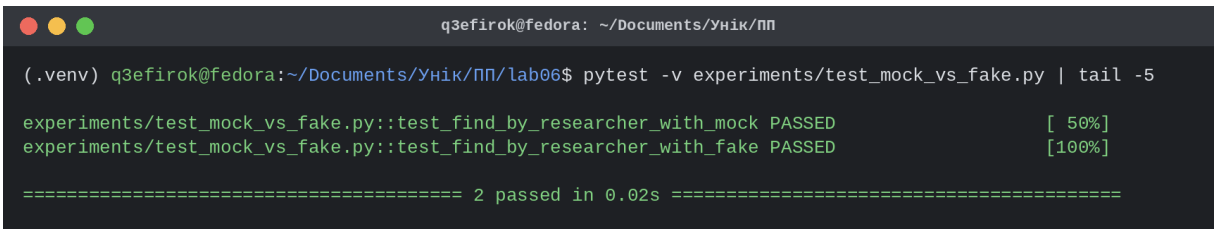


Рис. 6.14. Обидва варіанти тестів проходять

Таблиця 6.4. Порівняння Mock і fake

Критерій	Mock	Fake (InMemoryRepository)
Readability	Потрібно налаштувати return_value кожного методу	Звичайний код, як у production
State testing	Немає реального стану	Перевіряється результат пошуку за станом
Interaction testing	Зручно: `assert_called_once_with`	Незручно, потрібні додаткові лічильники
Reuse	Налаштування повторюється в кожному тесті	Одна реалізація для багатьох тестів
Coupling до	Високий: тест знає, який метод	Низький: перевіряє поведінку

Критерій	Mock	Fake (InMemoryRepository)
implementation	викликається	

5.5. Експеримент 5. Flaky test

Лістинг 6.22. Тест, залежний від випадкових значень

```
import random


def read_sensor() -> float:
    return random.gauss(300.0, 5.0)  # noisy temperature sensor


def test_sensor_reading_is_below_threshold() -> None:
    assert read_sensor() < 303.0
```

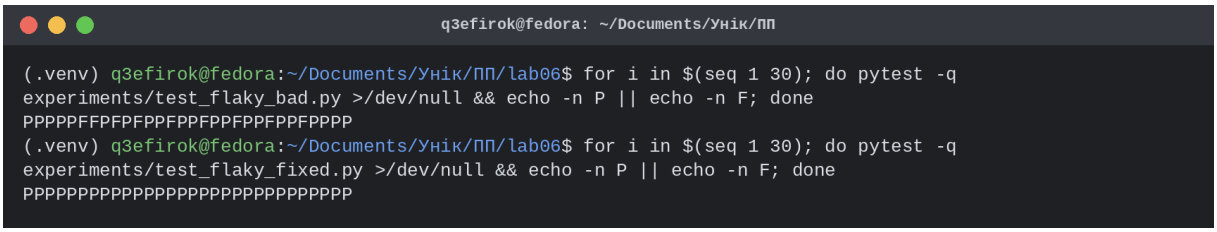


Рис. 6.15. 30 запусків нестабільного та детермінованого тесту

Нестабільний тест упав 9 разів із 30 без жодної зміни коду. Flaky tests небезпечні: їм перестають довіряти, справжні регресії ховаються серед «випадкових» падінь, CI стає непередбачуваним. Детермінована версія фіксує seed або підміняє джерело випадковості через `monkeypatch` і проходить у всіх запусках.

Лістинг 6.23. Детермінований варіант

```
def test_sensor_reading_with_monkeypatch(monkeypatch: pytest.MonkeyPatch) -> None:
    monkeypatch.setattr(random, "gauss", lambda mu, sigma: 301.5)
    assert read_sensor() < 303.0
```

5.6. Експеримент 6. tmp_path isolation

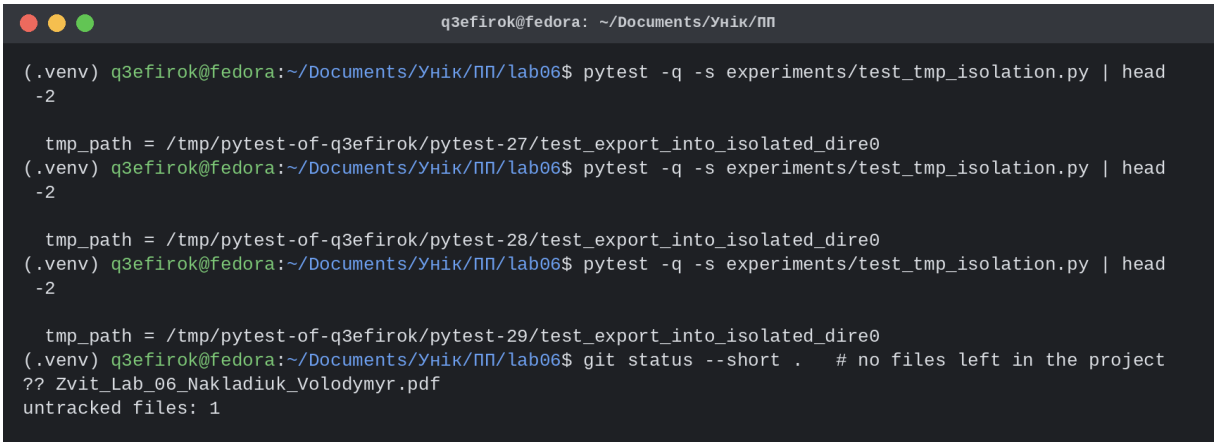


Рис. 6.16. Кожен запуск отримує новий каталог, у проєкті не з'являється файлів

Кожен запуск отримує окремий порожній каталог (`pytest-N`), а `git status` після тестів не показує нових файлів у проєкті — тести не залежать від попередніх запусків.

5.7. Експеримент 7. Failure path



Рис. 6.17. Сценарії успіху й збою залежності для ReportingService

Для сервісу із зовнішньою залежністю перевірено обидва сценарії: при успіху звіт передається в sink рівно один раз, при `side_effect=ExportError` виняток поширюється, а `sink.write` не викликається (`assert_not_called`) — немає небажаних взаємодій після збою. Аналогічно для асинхронного джерела перевірено, що збій одного експерименту не зупиняє збір інших.

5.8. Завдання підвищеної складності

- fixture factory `make_measurement` і fixture з teardown через `yield`;
- тест exception chaining (`__cause__`) і cleanup context manager `AtomicWriter`;
- test atomic export: попередній файл не пошкоджено, `.tmp` видалено;
- маркер `integration` і запуск груп тестів через `-m`;
- AsyncMock із перевіркою обмеження паралельності `Semaphore`;
- branch coverage з порогом `fail_under = 80` і HTML-звіт.

6. Відповіді на контрольні запитання

1. Що таке automated test?

Automated test — це код, який автоматично запускає певну частину програми із заданими вхідними даними та перевіряє, що фактичний результат відповідає очікуваному, без ручної участі людини. Такі тести можна повторювати будь-яку кількість разів, наприклад командою pytest локально або в CI.

2. Які основні переваги автоматизованого тестування?

Автоматизовані тести швидко й повторювано виявляють defects, захищають від regressions, дозволяють безпечно виконувати refactoring, документують очікувану поведінку та edge cases і контролюють contracts між компонентами. Вони запускаються в CI при кожній зміні, а разом із coverage дають об’єктивне уявлення про те, який код перевірено.

3. Що таке regression?

Regression — це дефект, за якого функціональність, що раніше працювала правильно, ламається після зміни коду: нової функції, виправлення іншого бага,

refactoring чи оновлення залежності. Набір regression tests, що запускається після кожної зміни, дозволяє виявити такий дефект одразу.

4. Що таке unit test?

Unit test перевіряє невелику одиницю поведінки (функцію, метод, клас) ізольовано від зовнішніх залежностей — файлової системи, мережі, БД, — які за потреби замінюють test doubles. Такі тести швидкі й детерміновані; наприклад, перевірка, що некоректний Measurement не створюється, або обчислення average value для ExperimentSeries.

5. Що таке integration test?

Integration test перевіряє коректність взаємодії декількох реальних компонентів разом, наприклад configuration loader, CSV reader, validator і JSON exporter. У проєкті integration test створює тимчасові CSV і YAML у tmp_path, запускає весь pipeline імпорту та експорту і перевіряє вміст отриманого JSON-файлу.

6. Чим integration test відрізняється від unit test?

Unit test перевіряє один компонент ізольовано, його залежності зазвичай замінено mocks або fakes, тому він швидкий і точно локалізує помилку. Integration test перевіряє кілька компонентів разом із реальними ресурсами (файли, парсери, SQLite) і виявляє проблеми на стиках між ними, але він повільніший і причину падіння в ньому знайти складніше.

7. Що показує test pyramid?

Test pyramid — модель співвідношення тестів: в основі найбільше швидких і дешевих unit tests, посередині менше integration tests, а на вершині найменше повільних і дорогих end-to-end tests. Вона показує, що зі зростанням рівня інтеграції тести дорожчають, повільнішають і стають крихкішими, тому їх має бути менше.

8. Чому unit tests зазвичай більше?

Unit tests швидкі, дешеві в написанні й підтримці, детерміновані та точно вказують місце помилки, тому ними зручно покривати багато сценаріїв, зокрема boundary та error cases. Integration та E2E tests повільніші, залежать від зовнішніх ресурсів і частіше бувають flaky, тому ними перевіряють лише ключові сценарії взаємодії.

9. Що таке pytest?

pytest — популярний фреймворк для тестування Python-коду, у якому тести пишуться як звичайні функції зі стандартним assert. Він має автоматичне discovery тестів, детальні повідомлення про failure, fixtures, parametrize, markers і багату екосистему плагінів, наприклад pytest-cov і pytest-asyncio.

10. Як pytest знаходить tests?

pytest рекурсивно обходить указані шляхи (або testpaths з конфігурації, або поточну директорію) і збирає файли test_*.py чи *_test.py. У них він знаходить функції з префіксом test та методи з префіксом test у класах Test* без методу __init__; у проєкті тести розміщено в tests/unit і tests/integration.

11. Якою командою запускається pytest?

pytest запускається з кореня проєкту командою pytest або python -m pytest; другий варіант додатково додає поточну директорію в sys.path. Можна вказати конкретну директорію, файл або тест, наприклад pytest tests/unit чи pytest tests/unit/test_models.py::test_name.

12. Для чого pytest -v?

Опція -v (verbose) вмикає детальний вивід: замість крапки для кожного тесту показується його повний ID (файл, функція, параметри) і статус PASSED, FAILED або SKIPPED. Це зручно, щоб бачити, які саме тести і з якими параметрами виконувалися.

13. Для чого pytest -x?

Опція -x (--exitfirst) зупиняє запуск після першого failure або error. Це корисно під час налагодження: можна зосередитися на першій помилці й не чекати виконання всього набору тестів.

14. Що означає Arrange–Act–Assert?

Arrange–Act–Assert — шаблон структури тесту з трьох частин: Arrange готує вхідні дані, об’єкти та залежності, Act виконує одну дію, що тестується, а Assert перевіряє результат. Наприклад: створити ExperimentSeries із кількома результатами, обчислити average value і порівняти його з очікуваним через pytest.approx.

15. Для чого використовується assert?

assert перевіряє, що умова істинна, і піднімає AssertionError, якщо вона хибна. pytest переписує assert-вирази (assertion rewriting) і при падінні показує детальні значення обох частин порівняння, тому спеціальні методи на кшталт assertEqual не потрібні.

16. Що таке assertion failure?

Assertion failure — ситуація, коли умова в assert виявилася хибною, тобто фактичний результат не відповідає очікуваному; виникає AssertionError, і pytest позначає тест як FAILED, показуючи значення та їхню різницю. Це відрізняється від error, коли тест падає через неочікуваний exception у коді чи fixture.

17. Чому floats не завжди слід порівнювати через ==?

Числа з плаваючою комою зберігаються у двійковому форматі IEEE 754 наближено, тому результати обчислень мають похибки округлення: наприклад, 0.1 + 0.2 == 0.3 дає False. Через це порівняння середніх значень вимірювань через == може падати, хоча результат математично правильний, тож порівнювати слід із допуском.

18. Для чого pytest.approx?

pytest.approx дозволяє порівнювати floating-point значення з допуском: за замовчуванням відносним 1e-6 і абсолютним 1e-12, або із заданими явно rel чи abs. Він працює і для списків, кортежів та словників чисел, наприклад assert average == pytest.approx(20.5).

19. Для чого pytest.raises?

pytest.raises — context manager, який перевіряє, що код усередині блоку with піднімає exception очікуваного типу або його підкласу; якщо винятку не було, тест

падає. Наприклад, with pytest.raises(RecordValidationError): для CSV-рядка з нечисловим result.

20. Як перевірити exception message?

Повідомлення перевіряють параметром match: pytest.raises(RecordValidationError, match="result"), де match — регулярний вираз, який шукається через re.search у str(exception). Для тексту зі спецсимволами варто застосовувати re.escape; також можна вручну перевірити str(exc_info.value).

21. Як отримати exception object через exc_info?

Конструкція with pytest.raises(SomeError) as exc_info: зберігає об’єкт ExceptionInfo, а сам exception доступний через exc_info.value (тип — через exc_info.type). Після блоку with можна перевірити атрибути, повідомлення чи причину винятку, наприклад isinstance(exc_info.value.__cause__, ValueError) для перевірки exception chaining.

22. Що таке boundary testing?

Boundary testing — техніка тестування значень на межах допустимих діапазонів і безпосередньо біля них (мінімум, максимум, трохи нижче й трохи вище), бо саме там найчастіше трапляються помилки на кшталт < замість <=. У проєкті так перевіряються межі result, задані в YAML-конфігурації.

23. Які boundaries потрібно тестувати для діапазону 0–100?

Для діапазону $0 \le x \le 100$ потрібно перевірити самі межі 0 і 100 (valid), значення одразу за межами, наприклад -0.01 і 100.01 (invalid), значення одразу всередині, наприклад 0.01 і 99.99, а також типове внутрішнє значення 50. Для цілих чисел це відповідно -1, 0, 1, 99, 100, 101.

24. Що таке parametrization?

Parametrization — запуск однієї тестової функції кілька разів з різними наборами вхідних даних і очікуваних результатів, причому кожен набір стає окремим тестом у звіті. Це прибирає дублювання однотипних тестів і робить перелік boundary та invalid випадків наочним.

25. Для чого pytest.mark.parametrize?

Декоратор @pytest.mark.parametrize задає імена аргументів тесту і список значень або кортежів для них, а pytest генерує окремий test case для кожного набору. Наприклад, @pytest.mark.parametrize(("value", "is_valid"), [(0, True), (100.01, False)]) перевіряє межі допустимого result одним тестом.

26. Що дають ids параметризованих tests?

ids задають зрозумілі назви наборів параметрів, які з’являються в ID тесту та виводі pytest, наприклад test_result[minimum] замість test_result[0-True]. Їх задають аргументом ids= або через pytest.param(..., id="minimum"); так легше зрозуміти, який саме випадок упав, і вибірково запустити його.

27. Що таке fixture?

Fixture — функція з декоратором @pytest.fixture, яка готує дані, об’єкти чи ресурси для тестів і за потреби прибирає їх після. Тест отримує fixture, просто вказавши її ім’я як параметр, а pytest автоматично викликає її через dependency injection.

28. Для чого використовуються fixtures?

Fixtures використовують для повторного використання підготовки тестового середовища (setup/teardown) без дублювання коду: створення domain objects, репозиторіїв, тимчасових файлів, конфігурацій, mocks. Наприклад, у conftest.py проєкту fixtures повертають готові Researcher чи Experiment або InMemoryRepository з тестовими даними.

29. Що означає fixture setup?

Fixture setup — частина fixture, що виконується перед тестом і готує потрібний стан: створює об’єкти, файли, з’єднання, заповнює дані. У fixture з yield це код до yield, а у звичайній fixture — увесь код до return.

30. Що означає fixture teardown?

Fixture teardown — завершальна частина fixture, що виконується після тесту, навіть якщо він упав, і звільняє ресурси: закриває з’єднання, видаляє файли, відновлює змінений стан. У pytest це код після yield або функції, зареєстровані через request.addfinalizer.

31. Для чого yield у fixture?

yield у fixture розділяє її на setup (код до yield) і teardown (код після yield), а значення після yield передається в тест. Після завершення тесту pytest продовжує виконання генератора, тому код після yield виконається навіть при падінні тесту, якщо setup пройшов успішно.

32. Які fixture scopes підтримує pytest?

pytest підтримує п’ять fixture scopes: function (для кожного тесту), class (раз на клас тестів), module (раз на файл), package (раз на пакет) і session (раз на весь запуск). Ширший scope економить час на дорогому setup, але спільний змінюваний стан може зробити тести залежними один від одного.

33. Який scope є default?

За замовчуванням fixture має scope="function", тобто створюється заново для кожного тесту, який її використовує, і знищується після нього. Це забезпечує максимальну ізоляцію тестів.

34. Для чого conftest.py?

conftest.py — спеціальний файл pytest для спільних fixtures, hooks і локальних плагінів, які автоматично стають доступними всім тестам у його директорії та піддиректоріях. У проєкті tests/conftest.py містить спільні fixtures для тестів у unit/ та integration/, наприклад типові domain objects і тестову конфігурацію.

35. Чому fixtures із conftest.py не імпортують вручну?

pytest сам знаходить і завантажує conftest.py під час collection та реєструє його fixtures, а тест отримує fixture за ім’ям параметра через dependency injection. Ручний

імпорт зайвий і навіть шкідливий: він може спричинити повторну реєстрацію, затінення fixtures і проблеми з імпортом, бо conftest.py не призначений бути звичайним модулем.

36. Що таке tmp_path?

tmp_path — вбудована fixture pytest, що надає унікальну тимчасову директорію для кожного тесту у вигляді pathlib.Path. Тест може створювати в ній файли, наприклад (tmp_path / "experiments.csv").write_text(..., encoding="utf-8"), а pytest сам керує розміщенням цих директорій і з часом прибирає старі.

37. Чому tmp_path корисний?

tmp_path дає ізольоване місце для файлових операцій, тому тести не засмічують робочу директорію проєкту, не перезаписують реальні дані й не впливають один на одного. Завдяки цьому можна писати integration tests з реальною файловою системою: створити тестові CSV і YAML, виконати імпорт і atomic export та перевірити JSON.

38. Що таке test isolation?

Test isolation — властивість, за якої кожен тест виконується незалежно: не залежить від порядку запуску, результатів інших тестів, спільного змінюваного стану чи зовнішнього середовища і не залишає після себе побічних ефектів. Її забезпечують fixtures з function scope, tmp_path, monkeypatch і test doubles.

39. Для чого monkeypatch?

monkeypatch — вбудована fixture pytest для тимчасової заміни атрибутів, функцій, методів, змінних середовища, елементів словників і sys.path під час тесту. Усі зміни автоматично скасовуються після його завершення, тому інші тести не зачіпаються; наприклад, нею можна зафіксувати поточну дату для детермінованого тесту.

40. Як monkeypatch змінює environment variable?

Виклик monkeypatch.setenv("NAME", "value") встановлює або перевизначає змінну середовища лише на час тесту, після чого попередній стан автоматично відновлюється. Так можна перевірити код, який читає налаштування зі змінних середовища, не змінюючи реальне оточення.

41. Як видалити environment variable у test?

monkeypatch.delenv("NAME") видаляє змінну середовища на час тесту; якщо змінної немає, за замовчуванням виникне KeyError, тому часто передають raising=False. Це дозволяє перевірити поведінку за її відсутності, наприклад використання значення за замовчуванням або підняття ConfigurationError.

42. Як підмінити function через monkeypatch?

Використовують monkeypatch.setattr(module, "function_name", fake_function) або форму з рядком-шляхом monkeypatch.setattr("package.module.function_name", fake_function). Підміняти потрібно ім’я в тому модулі, де воно використовується, а після тесту оригінал автоматично відновлюється.

43. Що таке test double?

Test double — загальна назва об’єкта, що замінює реальну залежність у тесті (мережу, БД, сервіс, джерело даних), щоб ізолювати тестований код і контролювати його оточення. Основні різновиди — Dummy, Stub, Fake, Mock і Spy; у Python їх часто створюють через unittest.mock.

44. Що таке Dummy?

Dummy — об’єкт, який передається лише для заповнення обов’язкового параметра, але фактично не використовується тестованим кодом. Наприклад, logger чи notifier, якого вимагає конструктор, хоча в сценарії, що перевіряється, він не задіяний.

45. Що таке Stub?

Stub — test double, що повертає заздалегідь задані відповіді на виклики, забезпечуючи тестованому коду потрібні вхідні дані, але не перевіряє, як саме його викликали. Наприклад, джерело вимірювань, яке завжди повертає фіксований список Measurement; у unittest.mock це Mock із return_value.

46. Що таке Fake?

Fake — спрощена, але робоча реалізація залежності, непридатна для production, наприклад in-memory repository замість справжньої БД. У проєкті роль fake може виконувати InMemoryRepository замість реального ExperimentRepository: він дійсно зберігає та знаходить об’єкти, і тест перевіряє кінцевий стан.

47. Що таке Mock?

Mock — test double, який записує всі виклики (аргументи, кількість, порядок) і дозволяє після дії перевірити, чи викликано потрібний метод із правильними параметрами, тобто перевіряє взаємодію. У Python це unittest.mock.Mock або MagicMock з методами на кшталт assert_called_once_with.

48. Що таке Spy?

Spy — test double, який записує інформацію про виклики (аргументи, кількість) для подальшої перевірки, зазвичай зберігаючи реальну поведінку об’єкта, що обгортається. У unittest.mock spy можна створити через Mock(wraps=real_object) або patch.object(..., wraps=...).

49. Для чого Mock?

Mock використовують, щоб ізолювати тестований код від зовнішніх чи повільних залежностей, задати їм потрібну поведінку (return_value, side_effect, зокрема збої) і перевірити взаємодію з ними. У проєкті Mock замінює analysis strategy або exporter для перевірки failure path, а AsyncMock — асинхронне джерело вимірювань.

50. Що таке spec?

spec — параметр Mock, наприклад Mock(spec=RealClass), який обмежує інтерфейс mock атрибутами й методами реального класу: звертання до неіснуючого атрибута піднімає AttributeError, а перевірка isinstance проходить. Це захищає від тестів, що проходять попри помилку в назві методу; create_autospec додатково перевіряє сигнатури викликів.

51. Для чого return_value?

return_value задає значення, яке mock повертає при виклику, наприклад dependency.method.return_value = 42. Так тест контролює вхідні дані для тестованого коду, імітуючи відповідь залежності без її реальної реалізації.

52. Для чого side_effect?

side_effect визначає поведінку mock при виклику: якщо це exception, mock піднімає його, що дозволяє тестувати failure path; якщо iterable — повертає значення по черзі при послідовних викликах; якщо функція — викликає її з тими самими аргументами. Наприклад, exporter із side_effect=ExportError(...) дає змогу перевірити обробку помилки експорту.

53. Як перевірити, що method викликано один раз?

Використовують mock.method.assert_called_once(), а з одночасною перевіркою аргументів — assert_called_once_with(...); якщо кількість викликів інша, тест падає з AssertionError. Також можна перевірити лічильник: assert mock.method.call_count == 1, наприклад щоб переконатися, що exporter викликано лише один раз.

54. Як перевірити parameters виклику?

Аргументи останнього виклику перевіряють через assert_called_with(...) або assert_called_once_with(...), наявність будь-якого виклику з такими аргументами — через assert_any_call(...), а послідовність викликів — через assert_has_calls([call(...), ...]). Можна також вручну аналізувати mock.call_args (args і kwargs) та call_args_list.

55. Як перевірити, що method не викликався?

Використовують mock.method.assert_not_called() або перевіряють assert mock.method.call_count == 0; якщо виклик був, тест упаде з повідомленням про фактичні виклики. Наприклад, у strict mode при невалідних вхідних даних exporter не повинен викликатися взагалі.

56. Що таке state testing?

State testing — перевірка кінцевого стану або результату після дії: повернутого значення, атрибутів об’єкта, вмісту репозиторію чи файлу. Наприклад, після додавання результату до Experiment перевіряють, що він з’явився серед його результатів, або що експортований JSON містить потрібні записи.

57. Що таке interaction testing?

Interaction testing — перевірка того, як тестований код взаємодіє із залежностями: які методи викликано, скільки разів, з якими аргументами й у якому порядку, зазвичай через mocks і assert_called_once_with. Наприклад, перевірка, що exporter викликано рівно один раз із валідними результатами.

58. Коли interaction testing виправдане?

Interaction testing виправдане, коли сама взаємодія є очікуваною поведінкою, а стан перевірити неможливо або недоцільно: надсилання повідомлень, виклик зовнішнього API, запис аудиту, виклик exporter, а також перевірка, що залежність не

викликається у failure path. Якщо важливий лише кінцевий результат, краще state testing, бо interaction tests крихкіші при refactoring.

59. Що таке patch?

patch з unittest.mock тимчасово замінює об’єкт за вказаним шляхом імпорту (рядок "package.module.name") на Mock на час тесту й автоматично відновлює оригінал. Його можна використовувати як декоратор, як context manager (with patch(...) as mocked:) або через patch.object для атрибута конкретного об’єкта.

60. Чому patch потрібно виконувати там, де name використовується?

patch замінює ім’я в просторі імен конкретного модуля, а не сам об’єкт усюди. Якщо модуль виконав from x import func, у нього є власне посилання func, і patch("x.func") його не змінить; тому патчити треба "module_that_uses.func", тобто там, де ім’я шукається під час виконання.

61. Що таке AsyncMock?

AsyncMock — клас unittest.mock для імітації асинхронних функцій і методів: його виклик повертає coroutine, який після await дає return_value або піднімає side_effect. У проєкті AsyncMock замінює асинхронне джерело вимірювань, що дозволяє тестувати async pipeline без реального I/O.

62. Як перевірити awaited call?

Для AsyncMock використовують assert_awaited(), assert_awaited_once(), assert_awaited_with(...), assert_awaited_once_with(...), assert_not_awaited() та атрибут await_count. Вони перевіряють саме очікування coroutine через await, а не лише виклик; асинхронний тест при цьому зазвичай запускають через pytest-asyncio з маркером @pytest.mark.asyncio.

63. Що таке flaky test?

Flaky test — тест, який за незмінного коду іноді проходить, а іноді падає. Типові причини — залежність від випадкових чисел, поточного часу, мережі, порядку виконання тестів, спільного стану або race conditions в асинхронному коді.

64. Чому flaky tests небезпечні?

Flaky tests підривають довіру до тестового набору: розробники звикають ігнорувати червоні збірки або просто перезапускати їх, і справжні regressions пропускаються. Вони також гальмують CI, забирають час на розслідування хибних падінь і можуть приховувати реальні проблеми зі станом чи конкурентністю.

65. Що таке deterministic test?

Deterministic test — тест, який за однакового коду завжди дає однаковий результат незалежно від часу, оточення, мережі чи порядку запуску. Для цього недетерміновані джерела (random, datetime.now(), UUID, мережу) контролюють через monkeypatch, фіксований seed або test doubles.

66. Чому unit test не повинен залежати від Internet?

Залежність від Internet робить unit test повільним, недетермінованим і неізольованим: мережа може бути недоступна, API може змінити відповідь або

обмежити запити, і тест упаде без жодної помилки в коді. Unit test має перевіряти лише власну логіку, тому зовнішні сервіси замінюють stubs, fakes чи mocks.

67. Коли в integration test доцільний real filesystem?

Real filesystem доцільний, коли тест перевіряє саме роботу з файлами: читання CSV з реальним кодуванням і newline, розбір YAML-конфігурації, atomic export через тимчасовий файл і os.replace, запис і повторне читання JSON. Для ізоляції такі тести працюють у tmp_path, а не в робочих директоріях проєкту.

68. Чому не потрібно mock-ати все?

Якщо замінити mocks усе, тест перевіряє не реальну поведінку, а лише те, що mocks викликали один одного, і може проходити при зламаному коді. Такі тести прив’язані до деталей реалізації та ламаються при refactoring; mocks варто застосовувати переважно на межах системи (мережа, час, зовнішні сервіси), а domain objects і стандартну бібліотеку використовувати реальні.

69. Що таке over-mocking?

Over-mocking — надмірне використання mocks, коли в тесті підмінено майже все (parser, validator, repository, model, logger), і він фактично перевіряє лише послідовність викликів між mocks. Наслідки — хибне відчуття безпеки, низька цінність тестів і їхня крихкість при будь-якій зміні реалізації.

70. Що таке coverage?

Coverage — метрика, що показує, яку частину коду (рядки, гілки) було виконано під час запуску тестів, зазвичай у відсотках. Вона допомагає знайти неперевірені ділянки, але сама не свідчить про якість перевірок; у Python її вимірює coverage.py, зокрема через плагін pytest-cov.

71. Що таке statement coverage?

Statement coverage (line coverage) — частка виконуваних інструкцій коду, які було виконано хоча б раз під час тестів. Вона не показує, чи перевірено обидва варіанти умови: if без else може бути повністю покритим, навіть якщо випадок хибної умови жодного разу не тестувався.

72. Що таке branch coverage?

Branch coverage — метрика, що показує, чи було пройдено кожен можливий перехід у коді: для кожного if, циклу чи іншого розгалуження мають виконатися як істинний, так і хибний варіант. У pytest-cov вона вмикається опцією --cov-branch або параметром branch = true у [tool.coverage.run], що використано в проєкті.

73. Чим вони відрізняються?

Statement coverage рахує виконані рядки, а branch coverage — пройдені переходи, тобто обидва результати кожної умови, тому вона суворіша. Наприклад, для if без else на кшталт if skip_invalid: logger.warning(...) тест лише зі значенням True виконає всі рядки (100% statement coverage), але branch coverage покаже, що перехід при False не перевірено.

74. Як отримати coverage через pytest-cov?

Потрібно встановити плагін (python -m pip install pytest-cov) і запустити pytest --cov=experiment_registry, що вимірює покриття пакета й виводить звіт у терміналі. Додатково використовують --cov-branch для branch coverage та --cov-report=term-missing або html для типу звіту, а постійні налаштування виносять у pyproject.toml у секції [tool.coverage.run] і [tool.coverage.report].

75. Для чого --cov-report=term-missing?

--cov-report=term-missing виводить звіт coverage у терміналі з додатковою колонкою Missing, де перелічено номери невиконаних рядків, а при branch coverage — і непройдені переходи на кшталт 12->15. Це дозволяє швидко побачити, які саме ділянки коду ще потребують тестів.

76. Для чого HTML coverage report?

--cov-report=html створює директорію htmlcov/ з інтерактивним звітом, який відкривається через htmlcov/index.html. У ньому вихідний код кожного файлу підсвічено кольорами — виконані, пропущені рядки та частково покриті гілки, — що зручно для детального аналізу великих модулів.

77. Чи гарантує 100% coverage відсутність bugs?

Ні. 100% coverage означає лише, що кожен рядок чи гілку було виконано, а не що результат перевірено правильно: тест без assert або з неправильними очікуваннями теж дає покриття. Coverage також не виявляє відсутньої функціональності, неперевірених комбінацій вхідних даних чи помилок у самих вимогах.

78. Чому meaningful assertions важливіші за coverage percentage?

Саме assertions перевіряють, що код поводиться правильно, а coverage лише фіксує, що рядки виконувалися. Тест із сильними перевірками конкретних значень, exceptions і стану виявить bug, тоді як тест «заради відсотка» без змістовних assert дасть високе покриття, але пропустить дефект; гонитва за метрикою якраз і провокує писати такі тести.

79. Що таке coverage threshold?

Coverage threshold — мінімально допустимий відсоток покриття, нижче якого запуск тестів вважається невдалим. У проєкті він задається як fail_under = 80 у секції [tool.coverage.report] файлу pyproject.toml (або опцією --cov-fail-under=80), тож CI не пропустить зміни, які знижують покриття нижче 80%.

80. Чому tests повинні бути незалежними один від одного?

Незалежні тести можна запускати в будь-якому порядку, окремо, вибірково чи паралельно, і вони дають стабільний результат. Якщо ж тести пов’язані спільним станом, файлами чи порядком виконання, падіння одного спричиняє каскад хибних падінь, ускладнює діагностику й породжує flaky tests; тому стан створюють через fixtures з function scope, tmp_path і monkeypatch.

7. Висновок

У лабораторній роботі для проєкту experiment-registry створено test suite з 157 тестів (з урахуванням параметризації): unit tests для value objects, сутностей, стратегій, репозиторію, сервісів, валідації, конфігурації, читачів і експорту, а також integration tests для імпорту, асинхронного конвеєра й консольного застосунку. Використано fixtures (у тому числі factory та з `yield`), parametrization, `pytest.raises`, `pytest.approx`, `tmp_path`, `monkeypatch`, `Mock`, `AsyncMock`, interaction assertions і маркери.

Загальне покриття з урахуванням гілок становить 92%, поріг 80% закріплено в конфігурації. Тести виявили два реальні дефекти (булеві рядки в YAML і `null` у JSON Lines), які виправлено й закріплено регресійними тестами. Експерименти показали, що виконання коду не є перевіркою, високий coverage не гарантує правильності, branch coverage точніше за statement coverage виявляє непротестовані умови, а flaky tests необхідно робити детермінованими.